MATERIAL DIDÁCTICO · PMDM

UNIDAD DIDÁCTICA 01

Relación 2
Repaso de Kotlin y Android

6 APLICACIONES · DIFICULTAD CRECIENTE · SIN ARRAYS

Funciones, validaciones, decisiones, interfaz XML, estilos y pequeños retos de lógica de negocio.

PROGRAMACIÓN MULTIMEDIA Y DISPOSITIVOS MÓVILES · 2.º DAM

CUADERNO DE ENUNCIADOS · BASES Y SOLUCIONES EN ZIP SEPARADAS

Guía del alumnado

Esta relación reúne todos los contenidos de la UD1 en aplicaciones más completas. Diseñarás la pantalla, conectarás vistas con Kotlin, validarás la entrada y comprobarás resultados. El código de partida es intencionalmente incompleto.

Qué debes saber antes de empezar

- Variables val y var; Int, Double, String y Boolean; operadores aritméticos y lógicos.
- Condicionales if/else, expresiones when y funciones con parámetros y return.
- EditText, TextView, Button, findViewById, setOnClickListener y conversión con toIntOrNull()/toDoubleOrNull().
- Diseño Android con interfaces XML (Views), no Jetpack Compose.

Novedades que se explican dentro de esta relación

- Estilos reutilizables en res/values/styles.xml y colores en res/values/colors.xml.
- Fondos XML redondeados en res/drawable y ScrollView cuando la pantalla no cabe.
- Uso básico de CheckBox y RadioGroup para opciones de usuario.
- Importes con dos decimales y cambio del color de un TextView desde Kotlin.

Condiciones de trabajo

- Cada ejercicio se construye en un proyecto Android Studio diferente y se prueba en un emulador o teléfono.
- No utilices arrays, listas, colecciones, bucles, bases de datos, servicios externos ni Jetpack Compose.
- Cada pantalla deberá tolerar entradas inválidas sin cerrarse ni efectuar operaciones erróneas.
- Las funciones de cálculo no deben depender de los widgets de la pantalla: reciben datos y devuelven resultados.
- Conserva al menos una captura con datos correctos y una con datos inválidos por ejercicio.

Relación de aplicaciones

Ejercicio	Aplicación	Contenido destacado
01	Registro de entrenamiento	Texto, rangos, funciones, estilos reutilizables
02	Pedido con gastos de envío	Decimales, condicionales, CheckBox, descomposición en funciones
03	Aparcamiento por tramos	Redondeo de tramos, RadioGroup, descuentos y límites
04	Boletín de evaluación	Ponderación, reglas conjuntas, when, estilos de resultado
05	Factura eléctrica simplificada	Tarifa progresiva, funciones, bonificaciones, impuestos
06	Reto final: reservas con aforo	Integración completa, estado entre pulsaciones y diseño XML

Mini tutorial: estilos en Android XML

En Android Views, el archivo activity_main.xml define qué controles existen. Los estilos reutilizan atributos de presentación, una idea parecida a una clase CSS, aunque no se escribe CSS. No se cambian las reglas de Kotlin ni la lógica del programa.

1. Crea colores reutilizables

En app/src/main/res/values/colors.xml añade, dentro de <resources>, colores con un nombre legible:

```
<color name="azul_principal">#19364F</color>
<color name="turquesa">#008E9B</color>
<color name="fondo_app">#F3F7FA</color>
```

2. Declara un estilo en styles.xml

Crea app/src/main/res/values/styles.xml, dentro de <resources>, y define un estilo para varios campos de texto. El nombre del estilo es CampoFormulario.

```
<style name="CampoFormulario">
    <item name="android:layout_width">match_parent</item>
    <item name="android:layout_height">wrap_content</item>
    <item name="android:textSize">16sp</item>
    <item name="android:padding">12dp</item>
    <item name="android:layout_marginBottom">8dp</item>
</style>
```

3. Utiliza el estilo en una vista

```
<EditText
    android:id="@+id/etMinutos"
    style="@style/CampoFormulario"
    android:hint="Minutos de entrenamiento"
    android:inputType="number" />
```

No repitas esos cinco atributos en cada EditText. En cambio, cada vista debe seguir teniendo su propio id y los atributos que la hacen específica (hint, inputType, text, etc.).

4. Medidas y apariencia

- dp: tamaños y márgenes de la interfaz. sp: tamaños de letra. match_parent: ocupa el espacio del contenedor. wrap_content: se ajusta al contenido.
- android:padding separa contenido y borde dentro de una vista; android:layout_marginTop o Bottom separa vistas entre sí.

Mini tutorial: detalles visuales e interacción

5. Un fondo redondeado sin imágenes

Crea app/src/main/res/drawable/fondo_panel.xml. Este archivo es un drawable de tipo shape:

```
<shape xmlns:android="http://schemas.android.com/apk/res/android"
    android:shape="rectangle">
    <solid android:color="@color/fondo_app" />
    <corners android:radius="16dp" />
    <stroke android:width="1dp" android:color="#CDDDE3" />
</shape>
```

Para aplicarlo a un TextView o un LinearLayout usa android:background="@drawable/fondo_panel" y añade padding al elemento para que el texto no toque el borde.

6. Más controles con XML y Kotlin

```
<CheckBox android:id="@+id/cbUrgente"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="Envío urgente" />

val urgente = findViewById<CheckBox>(R.id.cbUrgente).isChecked
```

Para ofrecer opciones excluyentes, mete varios RadioButton dentro de un RadioGroup. En Kotlin consulta checkedRadioButtonId: devuelve el id seleccionado o -1 si no se eligió ninguno.

7. Importes con dos decimales

```
import java.util.Locale
val importe = 12.5
val texto = String.format(Locale("es", "ES"), "%.2f €", importe)
// texto contendrá 12,50 €
```

8. Pantallas largas

Utiliza un ScrollView como raíz cuando haya muchos campos. Dentro debe contener un único hijo directo; normalmente un LinearLayout vertical con todos los controles.

Regla importante

Los estilos y el diseño ayudan a hacer la app más legible, pero no sustituyen las validaciones. Primero comprueba que los datos son correctos; después calcula y muestra el resultado.

EJERCICIO 01 / Registro de entrenamiento

Practicarás: Texto, rangos, funciones, estilos reutilizables

Qué tienes que construir

Diseña una ficha para registrar una sesión. El usuario introduce su nombre, la duración en minutos y una intensidad de 1 a 10. La app calcula una puntuación y clasifica el esfuerzo.

Elementos mínimos de la pantalla

etNombre (texto), etMinutos (entero), etIntensidad (entero), btnRegistrar, btnLimpiar, tvResultado.

Requisitos obligatorios

1. El nombre no puede estar vacío después de trim(). Minutos: entero de 1 a 300. Intensidad: entero de 1 a 10.
2. Crea fun calcularPuntos(minutos: Int, intensidad: Int): Int que devuelva minutos × intensidad.
3. Clasifica la intensidad con when: suave (1-3), moderada (4-6), intensa (7-10).
4. Muestra nombre, minutos, intensidad, categoría y puntos. El botón Limpiar restaura el mensaje inicial.
5. Define y utiliza en styles.xml al menos dos estilos reutilizables: CampoFormulario y BotonPrincipal.

Pista técnica para la interfaz

Un estilo agrupa atributos de aspecto: style="@style/CampoFormulario". Mantén los android:id individuales de cada elemento en activity_main.xml.

Casos que debes comprobar

Entrada / acción	Resultado esperado
Ana, 45, 2	Suave; 90 puntos.
Luis, 30, 7	Intensa; 210 puntos.
30, 0 o 11	Intensidad no válida.
Espacios, 30, 4	Nombre obligatorio.
Pulsar Limpiar	Campos vacíos, texto inicial.

Ampliación voluntaria

Añade una descripción específica de la categoría utilizando una función que devuelva String.

EJERCICIO 02 / Pedido con gastos de envío

Practicarás: Decimales, condicionales, CheckBox, descomposición en funciones

Qué tienes que construir

Una tienda necesita calcular el total de una compra a partir del importe de los artículos, su peso y si el cliente solicita envío urgente.

Elementos mínimos de la pantalla

etImporte (decimal), etPeso (decimal), cbUrgente, btnCalcular, btnLimpiar, tvResultado.

Requisitos obligatorios

1. Importe de productos: Double mayor o igual que 0. Peso: Double mayor que 0 y menor o igual que 30 kg.
2. Tarifa ordinaria por peso: hasta 1 kg, 3,50 €; más de 1 y hasta 5 kg, 6,50 €; más de 5 kg, 9,50 €.
3. En compras de 60 € o más, el envío ordinario es gratuito. El recargo urgente suma 4 € incluso si el envío ordinario es gratuito.
4. Usa fun costeEnvio(peso: Double, importe: Double, urgente: Boolean): Double. Muestra desglose y total.
5. Incorpora un CheckBox y lee su propiedad isChecked. Personaliza el botón de cálculo mediante un estilo.

Pista técnica para la interfaz

En XML: <CheckBox android:id="@+id/cbUrgente" ... />. En Kotlin: val urgente = findViewById<CheckBox>(R.id.cbUrgente).isChecked.

Casos que debes comprobar

Entrada / acción	Resultado esperado
20 €, 0,5 kg, normal	Envío 3,50 €; total 23,50 €.
80 €, 3 kg, normal	Envío 0 €; total 80 €.
80 €, 3 kg, urgente	Envío 4 €; total 84 €.
20 €, 6 kg, urgente	Envío 13,50 €; total 33,50 €.
Peso 0 o vacío	Error sin calcular.

Ampliación voluntaria

Añade un código de promoción ENVIO2 que reste 2 € de gastos de envío, sin permitir un envío negativo.

EJERCICIO 03 / Aparcamiento por tramos

Practicarás: Redondeo de tramos, RadioGroup, descuentos y límites

Qué tienes que construir

Construye una máquina de cobro de aparcamiento. Se paga por hora comenzada y el importe depende del vehículo y de si se aplica tarifa nocturna.

Elementos mínimos de la pantalla

etMinutos (entero), rgVehiculo con rbCoche y rbMoto, cbNocturno, btnCalcular, tvResultado.

Requisitos obligatorios

1. Admite entre 1 y 1.440 minutos, ambos incluidos; el usuario debe seleccionar coche o moto.
2. Cada hora iniciada se cobra completa. Calcula horas = (minutos + 59) / 60 utilizando división entera.
3. Coche: 1,80 €/hora y máximo diario de 15 €. Moto: 1,10 €/hora y máximo diario de 9 €.
4. Primero aplica el máximo diario y, después, si cbNocturno está marcado, un descuento del 20 %.
5. Crea una función para el importe base (antes del descuento) y muestra vehículo, horas, descuento y total.
6. Usa RadioGroup, RadioButton y una tarjeta de resultado con fondo XML de esquinas redondeadas.

Pista técnica para la interfaz

El grupo usa android:orientation="horizontal". En Kotlin, rgVehiculo.checkedRadioButtonId coincide con R.id.rbCoche o R.id.rbMoto; si vale -1 no hay selección.

Casos que debes comprobar

Entrada / acción	Resultado esperado
Coche, 60 min, día	1 hora; 1,80 €.
Coche, 61 min, día	2 horas; 3,60 €.
Moto, 600 min, día	10 horas; tope 9 €.
Coche, 600 min, noche	Tope 15 €; final 12 €.
Sin vehículo o 0 min	Mensaje de error.

Ampliación voluntaria

Añade un segundo TextView que explique qué regla se ha aplicado (tarifa normal o máximo diario).

EJERCICIO 04 / Boletín de evaluación

Practicarás: Ponderación, reglas conjuntas, when, estilos de resultado

Qué tienes que construir

Una asignatura se evalúa con tres bloques. La nota final es ponderada, pero aprobar exige tanto alcanzar la media mínima como superar el mínimo en cada bloque.

Elementos mínimos de la pantalla

etBloque1, etBloque2, etBloque3 (decimales), btnEvaluar, btnLimpiar, tvNota, tvEstado.

Requisitos obligatorios

1. Introduce tres notas Double entre 0 y 10, ambos incluidos. Rechaza campos vacíos y notas fuera de rango.
2. La ponderación es: bloque 1 = 20 %, bloque 2 = 30 % y bloque 3 = 50 %.
3. Se supera la asignatura cuando la media es >= 5 Y las tres notas son >= 4. En caso contrario, no superada.
4. Si no se supera, explica si el motivo es la media inferior a 5, algún bloque inferior a 4 o ambos.
5. Si se supera, muestra la calificación: Aprobado (< 7), Notable (< 9) o Sobresaliente (>= 9).
6. Usa una función para el cálculo de la media y destaca el estado en un TextView separado.

Pista técnica para la interfaz

Puedes cambiar el aspecto de un TextView con setTextColor(...) tras calcular, mientras que el aspecto fijo debe agruparse en un estilo XML.

Casos que debes comprobar

Entrada / acción	Resultado esperado
5, 5, 5	Media 5; superada, Aprobado.
10, 10, 3	Media 6,5; no superada: bloque < 4.
4, 4, 5	Media 4,5; no superada: media < 5.
3, 3, 4	Media 3,5; no superada: ambos motivos.
10, -1, 9	Notas no válidas.

Ampliación voluntaria

Muestra también la distancia respecto al 5, con valor positivo o negativo, sin modificar las reglas de superación.

EJERCICIO 05 / Factura eléctrica simplificada

Practicarás: Tarifa progresiva, funciones, bonificaciones, impuestos

Qué tienes que construir

Calcula una factura a partir del consumo en kWh y los días facturados. Esta vez debes separar correctamente el término fijo, la energía, los descuentos y el impuesto.

Elementos mínimos de la pantalla

etKwh (decimal), etDias (entero), cbBonoSocial, btnCalcular, btnLimpiar, tvResultado.

Requisitos obligatorios

1. Consumo válido: Double >= 0; días válidos: Int entre 1 y 31.
2. Término fijo: 0,45 € por día. Energía: primeros 100 kWh a 0,12 €/kWh y el exceso a 0,18 €/kWh.
3. Con bono social, descuenta un 25 % ÚNICAMENTE del coste de energía, no del término fijo.
4. Calcula base = fijo + energía bonificada. Añade después el 21 % de IVA sobre esa base.
5. Crea al menos dos funciones, una para el término de energía y otra para el total con IVA.
6. Muestra un desglose con al menos 5 conceptos y utiliza un estilo para diferenciar cifras y etiquetas.

Pista técnica para la interfaz

El tramo de consumo superior a 100 kWh se calcula como (kwh - 100) × 0,18. No apliques la tarifa superior a todos los kWh.

Casos que debes comprobar

Entrada / acción	Resultado esperado
100 kWh, 10 días, sin bono	Energía 12; fijo 4,50; total 19,97 €.
150 kWh, 10 días, sin bono	Energía 21; fijo 4,50; total 30,86 €.
150 kWh, 10 días, con bono	Energía tras bono 15,75; total 24,50 €.
0 kWh, 1 día, sin bono	Total 0,54 €.
Consumo -1 o 32 días	Error sin calcular.

Ampliación voluntaria

Añade una comparación: cuánto habría pagado el cliente sin bono social, si lo tiene activado.

EJERCICIO 06 / Reto final: reservas con aforo

Practicarás: Integración completa, estado entre pulsaciones y diseño XML

Qué tienes que construir

Crea una taquilla con aforo limitado. Debe permitir realizar reservas sucesivas mientras la app permanezca abierta, calcular precios y evitar que se vendan más entradas de las disponibles.

Elementos mínimos de la pantalla

etNombre, etCantidad, etCodigo; rgEntrada (rbGeneral, rbVip); cbFinSemana; btnReservar, btnLimpiar; tvResumen, tvPlazas.

Requisitos obligatorios

1. Al abrir, hay 20 plazas disponibles. El contador debe mantenerse entre clics en Reservar, pero no es necesario guardarlo al cerrar la app.
2. Nombre obligatorio y cantidad entera de 1 a 8. Debe elegirse tipo General (18 €/entrada) o VIP (32 €/entrada).
3. En fin de semana se aplica un recargo del 20 % al subtotal de entradas.
4. El código promocional opcional DAM10 descuenta el 10 % DESPUÉS del recargo, pero únicamente si se reservan 3 o más entradas. Otro código escrito es inválido.
5. No descuentas plazas ni actualizas el resumen si la entrada es inválida o se piden más plazas de las disponibles.
6. Muestra nombre, tipo, cantidad, base, recargo, descuento, total y plazas restantes. Usa funciones para los cálculos.
7. Limpiar borra los campos y el resumen, pero NO recupera plazas ni cancela reservas ya realizadas.
8. La pantalla debe desplazarse con ScrollView; usa estilos, colores XML y drawable/fondo_panel.xml.

Pista técnica para la interfaz

Declara var plazasDisponibles = 20 en onCreate, fuera del listener. Para limpiar, editText.setText("") y radioGroup.clearCheck(). Un ScrollView solo puede tener un hijo directo.

Casos que debes comprobar

Entrada / acción	Resultado esperado
Ana, 2 General, laborable, sin código	Total 36 €; quedan 18 plazas.
Luis, 3 VIP, fin de semana, DAM10	Base 96 €; recargo 19,20 €; descuento 11,52 €; total 103,68 €.
Tras las dos reservas anteriores	Quedan 15 plazas.
Pedir 8 cuando quedan 7	Rechaza y conserva 7 plazas.
DAM10 con 2 entradas	Error; plazas sin cambios.
Pulsar Limpiar	Campos vacíos; aforo se mantiene.

Ampliación voluntaria

Añade una etiqueta que cambie a «Últimas plazas» si quedan 5 o menos, sin utilizar arrays ni bases de datos.